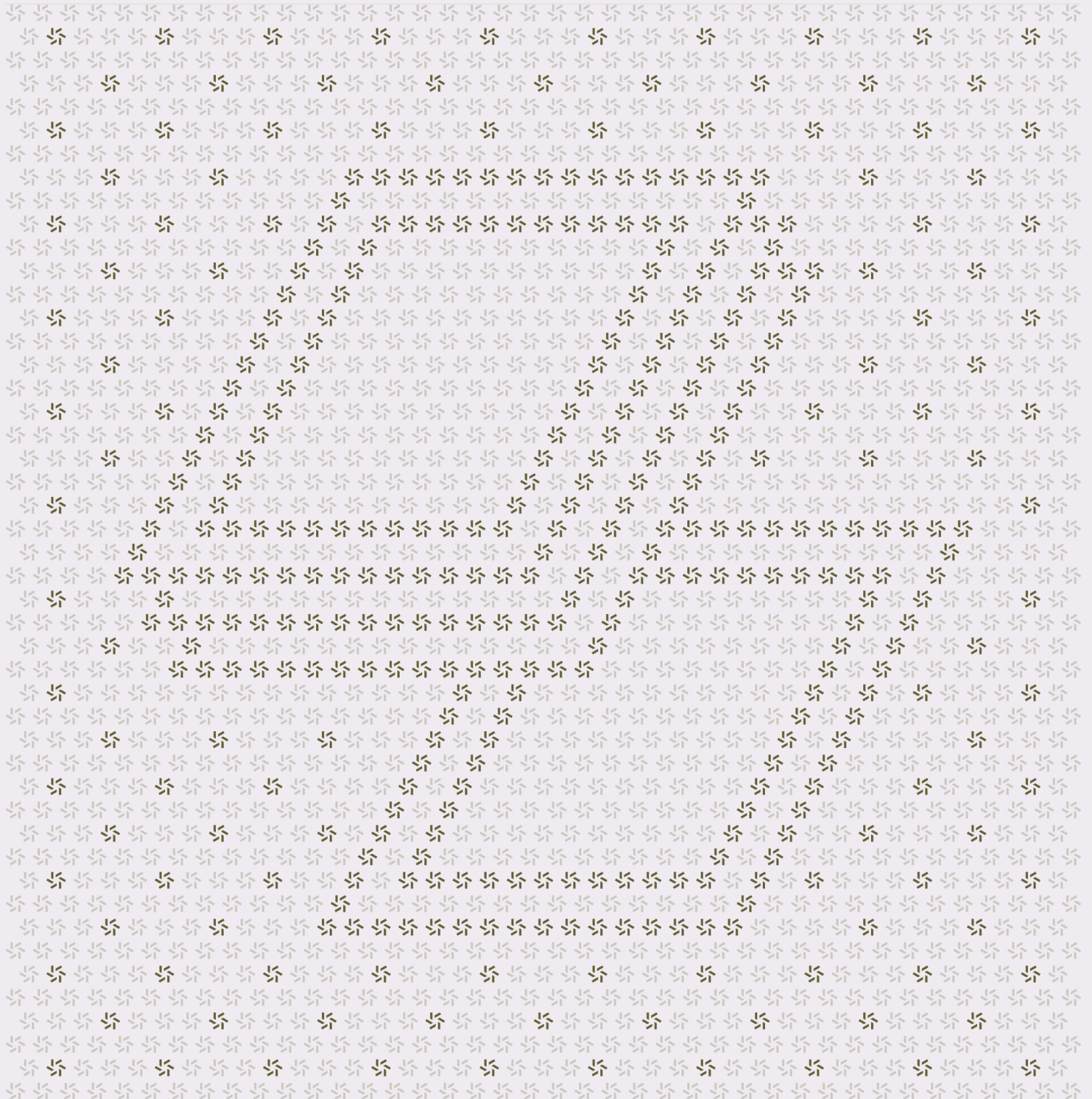


July 23, 2026

OpenZeppelin Cairo Contracts

Cairo Application Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	7
2.1. About OpenZeppelin Cairo Contracts	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	10
2.5. Project Timeline	11
3. Detailed Findings	12
3.1. No missing-hook warning for the ERC6909TokenSupply component	12
3.2. Spurious "missing ImmutableConfig" warning emitted by ERC20FlashMint	14
3.3. No warning from Votes when transfer_voting_units hook wiring is missing	16
3.4. Initializer detection misses alternate (no-metadata) initializers	18
3.5. Reserved SNIP-12 names silently shadow user-defined types	20
3.6. Only the first #[snip12] attribute is honored; bare #[snip12] is a silent no-op	22
3.7. Duplicate keys inside one #[snip12(...)] are silently last-write-wins	24

3.8.	Duplicate emitted member/variant names are not detected	26
3.9.	User-provided names are not JSON-escaped in the encoded type	28
3.10.	SafeERC20 has no camel-case fallback for non-snake-case tokens	30
3.11.	The <code>type_hash</code> macro does not support the <code>bool</code> and <code>string</code> basic types	32
3.12.	Substring-based trait/hook detection produces false warnings	34
3.13.	Enum variant encoding omits the colon used by common SNIP-12 implementations	36
3.14.	Tuple member encoding is not specified by SNIP-12	38
3.15.	The <code>assert_increase_allowance</code> function reverts when the current allowance is unlimited	39
3.16.	The <code>ERC20FlashMint</code> default configuration permits unbounded flash-mint amounts	41
3.17.	Inconsistent supply-tracking extension strategy across components	43
4.	Discussion	45
4.1.	Notable changes	45
5.	Assessment Results	47
5.1.	Disclaimer	47

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and](#) follow [@zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for OpenZeppelin from July 6th to July 21st, 2026. During this engagement, Zellic reviewed OpenZeppelin Cairo Contracts's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is the ERC20FlashMint implementation secure against inflation or supply accounting bugs?
 - Can a borrower complete a flash mint without returning the full minted amount plus the required fee?
 - Are balances, total supply, and allowances always restored or updated correctly after a flash mint?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

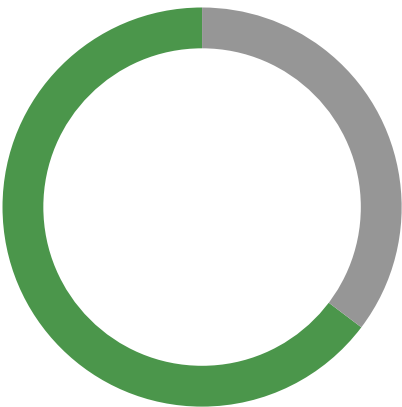
1.4. Results

During our assessment on the scoped OpenZeppelin Cairo Contracts targets, we discovered 17 findings. No critical issues were found. Eleven findings were of low impact and the remaining findings were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of OpenZeppelin in the Discussion section ([4.7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	11
Informational	6



2. Introduction

2.1. About OpenZeppelin Cairo Contracts

OpenZeppelin contributed the following description of OpenZeppelin Cairo Contracts:

OpenZeppelin Contracts for Cairo is a library of reusable, secure, and community-audited smart contract components for building applications on Starknet. The project provides standard implementations of common patterns such as tokens, access control, governance, and utilities, with a focus on security, composability, and developer experience.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the targets.

Architecture risks. This encompasses potential hazards originating from the blueprint of a system, which involves its core validation mechanism and other architecturally significant constituents influencing the system's fundamental security attributes, presumptions, trust mode, and design.

Arithmetic issues. This includes but is not limited to integer overflows and underflows, floating-point associativity issues, loss of precision, and unfavorable integer rounding.

Implementation risks. This encompasses risks linked to translating a system's specification into practical code. Constructing a custom system involves developing intricate on-chain and off-chain elements while accommodating the idiosyncrasies and challenges presented by distinct programming languages, frameworks, and execution environments.

Availability. Denial-of-service attacks are another leading issue in custom systems. Issues including but not limited to unhandled panics, unbounded computations, and incorrect error handling can potentially lead to consensus failures.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped targets itself. These observations — found in the Discussion ([4.7](#)) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

OpenZeppelin Cairo Contracts Targets

Types	Cairo, Rust
Platform	Starknet
Target	Only the changes between ddd3338...abfec31 (changes from v3.0.0 to v4.0.0-rc.1)
Repository	https://github.com/OpenZeppelin/cairo-contracts ↗
Version	abfec311e064faf15c8b0b05dddd130612f29a2f
Programs	packages/**/*. excluding packages/macros/*
Target	Only the changes between f1c15a3...abfec31 (changes from v2.0.0 to v4.0.0-rc.1)
Repository	https://github.com/OpenZeppelin/cairo-contracts ↗
Version	abfec311e064faf15c8b0b05dddd130612f29a2f
Programs	packages/macros/src/attribute/with_components/*.rs

Target	type_hash macro
Repository	https://github.com/OpenZeppelin/cairo-contracts ↗
Version	abfec311e064faf15c8b0b05dddd130612f29a2f
Programs	packages/macros/src/attribute/type_hash/*.rs

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 3.3 person-weeks. The assessment was conducted by two consultants over the course of 2.4 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
↗ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Jinseo Kim
↗ Engineer
jinseo@zellic.io ↗

Quentin Lemauf
↗ Engineer
quentin@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

July 6, 2026	Kick-off call
---------------------	---------------

July 6, 2026	Start of primary review period
---------------------	--------------------------------

July 21, 2026	End of primary review period
----------------------	------------------------------

3. Detailed Findings

3.1. No missing-hook warning for the ERC6909TokenSupply component

Target	with_components		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

Like several other extensions, ERC6909TokenSupplyComponent only tracks state correctly if the integrator manually calls its `update_token_supply` function from an ERC6909HooksTrait hook. The `with_components` macro warns when such wiring is missing for ERC1155Supply, ERC721Enumerable, ERC721URIStorage, and ERC721Consecutive, but `add_per_component_warnings` has no arm for ERC6909TokenSupply; it falls through to the default `_ => {}`:

```
AllowedComponents::ERC1155Supply => {
    let hook_called = code.contains("erc1155_supply.after_update(")
    || code.contains("ERC1155SupplyInternalImpl::after_update(");
    if !hook_called {
        let warning =
Diagnostic::warn(warnings::ERC1155_SUPPLY_HOOKS_MISSING);
        warnings.push(warning);
    }
}
// ... ERC721Enumerable, ERC721URIStorage, ERC721Consecutive arms ...
// no ERC6909TokenSupply arm
_ => {}
```

Because of this, a contract using `#[with_components(ERC6909, ERC6909TokenSupply)]` together with the empty hooks impl compiles and deploys with no diagnostic at all, while `total_supply(id)` silently stays 0 as tokens are minted and burned.

Impact

Any downstream consumer relying on `total_supply(id)` (indexers, DeFi integrations, governance weight) reads a permanently zero supply, with no compile-time hint that the required hook wiring is missing. The contract also advertises IERC6909TokenSupply support through SRC5, so the breakage is not observable from the interface.

Recommendations

Add an arm for AllowedComponents::ERC6909TokenSupply analogous to the ERC1155Supply one:

```
AllowedComponents::ERC6909TokenSupply => {  
    let hook_called =  
    code.contains("erc6909_token_supply.update_token_supply()"  
    ||  
    code.contains("ERC6909TokenSupplyInternalImpl::update_token_supply()");  
    if !hook_called {  
        let warning =  
        Diagnostic::warn(warnings::ERC6909_TOKEN_SUPPLY_HOOKS_MISSING);  
        warnings.push(warning);  
    }  
}
```

Remediation

This issue has been acknowledged by OpenZeppelin, and a fix was implemented in [PR #1733](#). They intend to merge these changes into the production branch.

3.2. Spurious "missing ImmutableConfig" warning emitted by ERC20FlashMint

Target	with_components		
Category	Coding Mistakes	Severity	Low
Likelihood	Medium	Impact	Low

Description

The ERC20FlashMintComponent does not define an ImmutableConfig. Its compile-time configuration is provided through FlashMintConfigTrait (max_flash_loan, flash_fee, flash_fee_receiver). However, its ComponentInfo entry sets has_immutable_config: true:

```
AllowedComponents::ERC20FlashMint => ComponentInfo {
  name: "ERC20FlashMintComponent",
  path: "openzep-
  pelin_token::erc20::extensions::erc20_flash_mint::ERC20FlashMintComponent",
  storage: "erc20_flash_mint",
  event: "ERC20FlashMintEvent",
  has_initializer: false,
  has_immutable_config: true,
  internal_impls: vec![],
},
```

The validate_contract_module iterates every component with has_immutable_config and looks for a DefaultConfig import or an ImmutableConfig impl. Since ERC20FlashMint never ships either (it uses FlashMintConfigTrait), the search always fails and the macro raises IMMUTABLE_CONFIG_MISSING even for a fully correct contract.

Impact

Every well-formed contract that uses ERC20FlashMint receives a false warning telling the developer that ImmutableConfig is missing. This trains developers to ignore the macro's warnings, which weakens the value of the legitimate warnings.

Recommendations

Set has_immutable_config: false for ERC20FlashMint. If the intent is to remind integrators to configure FlashMintConfigTrait, add a dedicated, correctly named check in add_per_component_warnings instead of reusing the immutable-config path.

Remediation

This issue has been acknowledged by OpenZeppelin, and a fix was implemented in [PR #1743](#). They intend to merge these changes into the production branch.

3.3. No warning from Votes when transfer_voting_units hook wiring is missing

Target	with_components		
Category	Integration Risks	Severity	Low
Likelihood	Low	Impact	Low

Description

A token component that carries voting units (ERC20 + Votes, ERC721 + Votes) keeps its vote accounting in sync only if the integrator calls `self.votes.transfer_voting_units(...)` from the token's update hook. The Votes arm of `add_per_component_warnings` checks only for `SNIP12Metadata`, not for the hook wiring:

```
AllowedComponents::Votes => {
  // Check that the SNIP12Metadata is implemented
  let snip12_metadata_implemented = code.contains("of SNIP12Metadata");
  if !snip12_metadata_implemented {
    let warning =
      Diagnostic::warn(warnings::SNIP12_METADATA_IMPL_MISSING);
    warnings.push(warning);
  }
}
```

As a result, the following contract compiles with `Diagnostics: None` even though vote accounting is never updated:

```
#[with_components(ERC20, Votes, Nonces)]
pub mod MyToken {
  use openzeppelin_token::erc20::{DefaultConfig, ERC20HooksEmptyImpl};
  use openzeppelin_utils::cryptography::snip12::SNIP12Metadata;
  // no votes.transfer_voting_units(...) hook wiring
}
```

Two special cases of this gap were also observed:

1. **ERC721Consecutive + Votes** — batch minting through `mint_consecutive` does not route through `update`, so `transfer_voting_units` must be invoked manually; nothing warns about it.
2. **ERC20FlashMint + Votes** — the `ERC20FlashMint` docs advise against combining with

Votes, but the macro does not surface this combination as a warning.

Impact

The `num_checkpoints`, `get_votes`, and delegation totals stay at zero (or diverge from balances) while the contract appears fully functional, silently corrupting any governance decision that relies on them.

Recommendations

Warn when `Votes` is combined with `ERC20` or `ERC721` without `transfer_voting_units()` call is detected in the module body. Additionally, warn when `Votes` is combined with `ERC721Consecutive` or `ERC20FlashMint`, which needs special attention for integrating them correctly.

Remediation

This issue has been acknowledged by OpenZeppelin, and a fix was implemented in [PR #1734](#). They intend to merge these changes into the production branch.

3.4. Initializer detection misses alternate (no-metadata) initializers

Target	with_components		
Category	Coding Mistakes	Severity	Low
Likelihood	Medium	Impact	Low

Description

The `validate_contract_module` decides whether a component's initializer was called by looking for the exact string `self.<storage>.initializer()`:

```
for component in components_with_initializer.iter() {
    if !constructor_code.contains(&format!("self.{}.initializer(",
        component.storage)) {
        components_with_initializer_missing.push(component.short_name());
    }
}
```

Several components expose alternate initializers that are equally valid. In particular, ERC721 (and ERC1155) provide a documented no-metadata path, `initializer_no_metadata`. A constructor that legitimately uses it

```
#[constructor]
fn constructor(ref self: ContractState) {
    self.erc721.initializer_no_metadata();
}
```

still fails the `self.erc721.initializer()` substring check and triggers a false `INITIALIZERS_MISSING` warning for ERC721.

Impact

Contracts following a documented initialization path are flagged as misinitialized. As with the other false-positive warnings, this erodes trust in the macro's diagnostics.

Recommendations

Teach the initializer check about the alternate initializers (at minimum `initializer_no_metadata` for ERC721 / ERC1155) — for example by accepting `self.<storage>.initializer` as a prefix and

matching any of the known initializer method names for that component.

Remediation

This issue has been acknowledged by OpenZeppelin, and a fix was implemented in [PR #1732](#). They intend to merge these changes into the production branch.

3.5. Reserved SNIP-12 names silently shadow user-defined types

Target	type_hash		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The `S12Type::from_str` function matches reserved basic, preset, and domain names before falling through to the `Custom(...)` catch-all:

```
Some(match s {
    "felt252" => S12Type::Basic(BasicType::Felt),
    // ... other basic types ...
    "TokenAmount" => S12Type::Preset(PresetType::TokenAmount),
    "NftId" => S12Type::Preset(PresetType::NftId),
    "u256" => S12Type::Preset(PresetType::U256),
    "StarknetDomain" => S12Type::UserDefined(UserDefinedType::StarknetDomain),
    _ => S12Type::UserDefined(UserDefinedType::Custom(s.to_string())),
})
```

This produces two related shadowing gaps, neither of which emits a diagnostic:

1. **Member type shadowing.** If an author defines their own type using a reserved name and references it inside a `#[type_hash]` type, the member is encoded as the SNIP-12 preset, not the author's type:

```
struct NftId {
    pub custom_id: felt252,
}

#[type_hash]
struct Order {
    pub item: NftId,
}
```

Here, `item` is encoded as the preset

`"NftId"("collection_address":"ContractAddress","token_id":"u256")` instead of `"NftId"("custom_id":"felt")`. A genuine (nonreserved) custom type fails closed with `CUSTOM_TYPE_NOT_SUPPORTED`; only reserved names bypass that guard.

2. **Primary type name.** The `#[type_hash(name: "...")]` is interpolated verbatim as the

top-level container name and never validated, and `#[ type_hash ( name: "u256" ) ]`
`struct Foo { a: u256 }` produces

```
"u256" ( "a": "u256" ) "u256" ( "low": "u128", "high": "u128" )
```

which contains the reserved name `u256` twice with two different roles.

Impact

The generated `*_TYPE_HASH` no longer describes the real runtime structure of the message: the affected parent type hashes to a spec-noncompliant value. A spec-compliant signer/wallet computing the hash from the type definition produces a different result, so signatures no longer interoperate. (This yields a wrong hash for the shadowed type; it is not a reachable cross-type collision, since the only encodable referenced types are the fixed presets.)

Recommendations

Reject reserved SNIP-12 names when they are used as a member's type or as the primary `#[ type_hash ( name: . . . ) ]` value, emitting a diagnostic error instead of silently accepting them. SNIP-12 explicitly forbids user-defined types from reusing reserved names.

Remediation

This issue has been acknowledged by OpenZeppelin, and a fix was implemented in [PR #1738](#). They intend to merge these changes into the production branch.

3.6. Only the first #[snip12] attribute is honored; bare #[snip12] is a silent no-op

Target	type_hash		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The `get_name_and_type_from_attributes` function returns on the first attribute recognized as `snip12(...)`, ignoring any later ones:

```
for attribute in attributes {
    let attribute_text =
        attribute.as_syntax_node().get_text_without_trivia(db);
    let Some(arguments)
        = snip12_attribute_arguments(attribute_text.long(db).as_str()) else {
        continue;
    };
    return parse_snip12_args(arguments);
}
```

If an author reasonably splits the override across two attributes, only the first is applied:

```
#[snip12(name: "ts")]
#[snip12(kind: "timestamp")] // silently dropped
member: u128,
```

The kind override is lost, so the field is encoded from its real Cairo type (`"ts": "u128"`) instead of the intended `"ts": "timestamp"`.

Separately, a bare `#[snip12]` with no arguments is accepted as an empty override that does nothing, since `snip12_attribute_arguments` returns `Some("")` for the empty argument list and `parse_snip12_args` treats an empty string as the default args. An override with a typo therefore silently has no effect.

Impact

The emitted SNIP-12 schema diverges from what the author intended, so the generated type hash disagrees with a spec-compliant signer and signature interoperability breaks — with no diagnostic

to indicate that part of the override was ignored.

Recommendations

Error when more than one `#[snip12( . . . )]` attribute is present on a single member (or merge them), and reject a bare `#[snip12]` with no arguments rather than accepting it as a silent no-op.

Remediation

This issue has been acknowledged by OpenZeppelin, and a fix was implemented in [PR #1736](#). They intend to merge these changes into the production branch.

3.7. Duplicate keys inside one `#[snip12(...)]` are silently last-write-wins

Target	type_hash		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The `parse_snip12_args` function iterates the comma-separated arguments and assigns each recognized key without checking whether it was already set:

```
for arg in parts {
    let Some((name, value)) = arg.split_once(':') else {
        return
    }
    Err(Diagnostic::error(errors::INVALID_SNIP12_ATTRIBUTE_FORMAT));

    match name.trim() {
        "name" => args.name = parse_string_arg(value.trim())?,
        "kind" => args.kind = parse_string_arg(value.trim())?,
        _ =>
        return Err(Diagnostic::error(errors::INVALID_SNIP12_ATTRIBUTE_FORMAT)),
    }
}
```

So a duplicated key is accepted, and the last occurrence wins:

```
#[snip12(name: "first", name: "second")]
pub member: felt252,
```

The effective override becomes `name = "second"` with no diagnostic. This is inconsistent with `#[type_hash(...)]` argument parsing, which already rejects duplicate keys.

Impact

A contradictory attribute silently resolves to one of the two values, which may not be the one the author expects; the generated type hash then differs from intent with no warning. The impact is limited to authors who write malformed attributes.

Recommendations

Reject duplicate keys within a single `#[snip12(...)]` attribute, matching the behavior already implemented for `#[type_hash(...)]`.

Remediation

This issue has been acknowledged by OpenZeppelin, and a fix was implemented in [PR #1737](#). They intend to merge these changes into the production branch.

3.8. Duplicate emitted member/variant names are not detected

Target	type_hash		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

When building the encoded type string, the parser appends each member's final SNIP-12 name without tracking whether that name was already emitted:

```
match self.plugin_type_info.type_variant {
  TypeVariant::Struct => {
    encoded_type.push_str(&format!("{name}\": \"{type_name}\""))
  }
  TypeVariant::Enum => {
    let tuple = maybe_tuple(&type_name)?;
    encoded_type.push_str(&format!("{name}({})", name, tuple))
  }
};
```

Because the final name can come from a `#[snip12(name: ...)]` override, two distinct members can end up with the same emitted name:

```
#[type_hash]
struct Msg {
  #[snip12(name: "x")]
  a: u128,
  x: u128,
}
```

This is accepted and emits an encoded type that contains the key "x" twice, with no diagnostic.

Impact

A duplicated key produces an ambiguous, noncanonical SNIP-12 schema. A spec-compliant signer/wallet may reject it or resolve it differently, breaking signature interoperability.

Recommendations

Track the final emitted member/variant names during parsing, and emit a diagnostic error if a duplicate would be produced.

Remediation

This issue has been acknowledged by OpenZeppelin, and a fix was implemented in [PR #1740](#). They intend to merge these changes into the production branch.

3.9. User-provided names are not JSON-escaped in the encoded type

Target	type_hash		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

SNIP-12 revision 1 defines `escape(name)` as the double-quoted name "with any escaping applied, following the spec for JSON objects" — a literal quote inside the name must be backslash-escaped, and a literal backslash must be doubled. The macro decodes escape sequences in `#[snip12(name: "...")] / #[type_hash(name: "...")]` values (via `decode_escaped_string`) but interpolates the decoded string into the type string with only surrounding quotes added:

```
TypeVariant::Struct => {  
    encoded_type.push_str(&format!("{}", {name}\": \"{type_name}\"",))  
}
```

The primary type name and enum variant names are interpolated the same way. There is no reescaping step. Type names legal in Cairo use only ASCII alphanumerics and `_`, so this is adequate for names derived from Cairo identifiers — but an overridden name may contain arbitrary characters:

```
#[type_hash]  
struct MyType {  
    #[snip12(name: "field\"name")]  
    member: felt252,  
}
```

The escaped quote in the override is decoded to a bare quote, so the emitted type string becomes the following:

```
"MyType"("field"name":"felt")
```

This is a malformed string that no spec-compliant wallet reproduces. Because the decoded name is spliced in raw, a crafted name can also inject structure (for example, a name that decodes to `x": "felt", "y)`, letting two structurally distinct types emit an identical type-hash string.

Impact

An overridden name containing a quote or a backslash yields a type-hash string that diverges from what a spec-compliant signer computes (signature verification fails), and in the crafted case enables a deliberate type-hash collision between distinct types.

Recommendations

After decoding, reapply JSON escaping — emit a quote as an escaped quote and a backslash as a doubled backslash — to every user-provided name (primary type name, field names, and enum variant names) before interpolating them into the type-hash string.

Remediation

This issue has been acknowledged by OpenZeppelin, and a fix was implemented in [PR #1741](#). They intend to merge these changes into the production branch.

3.10. SafeERC20 has no camel-case fallback for non-snake-case tokens

Target	SafeERC20		
Category	Integration Risks	Severity	Low
Likelihood	Low	Impact	Low

Description

On Starknet, some deployed ERC-20 tokens still expose only the camel-case entry points (transferFrom, balanceOf, ...) and do not implement the snake case methods; SafeERC20DispatcherTrait dispatches exclusively through IERC20Dispatcher, which calls the snake-case selectors:

```
fn assert_transfer_from(
    self: IERC20Dispatcher, sender: ContractAddress, recipient:
    ContractAddress, amount: u256,
) {
    assert(self.transfer_from(sender, recipient, amount),
    Errors::SAFE_ERC20_FAILED_OPERATION);
}
```

Because a Cairo dispatcher panics when the callee has no matching entry point, calling any SafeERC20 helper against a camel case-only token reverts. Developers reasonably expect a "safe" wrapper to abstract over this ecosystem quirk, but there is no fallback to the camel-case selector.

Impact

Contracts that route token operations through SafeERC20 are unusable with camel-case-only tokens; every transfer/approve reverts. This restricts the set of tokens an integrating protocol can support, without any indication at integration time.

Recommendations

Either add a camel-case fallback (attempt the snake-case selector and, on entry-point failure, retry the camel-case equivalent, as the community DualCase-style dispatchers do), or explicitly document that SafeERC20 supports only snake-case-conforming tokens so integrators can make an informed choice.

Remediation

This issue has been acknowledged by OpenZeppelin. OpenZeppelin notified us that they decided to not support camel-case methods as part of their long-term effort of deprecating camel-case interfaces on ERC-20 tokens.

3.11. The type_hash macro does not support the bool and string basic types

Target	type_hash		
Category	Completeness	Severity	Low
Likelihood	Low	Impact	Low

Description

SNIP-12 defines bool and string as basic types, but BasicType and S12Type::from_str do not list them:

```
Some(match s {
    "felt252" => S12Type::Basic(BasicType::Felt),
    "shortstring" => S12Type::Basic(BasicType::ShortString),
    "ClassHash" => S12Type::Basic(BasicType::ClassHash),
    "ContractAddress" => S12Type::Basic(BasicType::ContractAddress),
    "timestamp" => S12Type::Basic(BasicType::Timestamp),
    "selector" => S12Type::Basic(BasicType::Selector),
    "merkletree" => S12Type::Basic(BasicType::MerkleTree),
    "u128" => S12Type::Basic(BasicType::U128),
    "i128" => S12Type::Basic(BasicType::I128),
    // no "bool", no "string"
    // ...
    _ => S12Type::UserDefined(UserDefinedType::Custom(s.to_string())),
})
```

A bool field (or a ByteArray field annotated #[snip12(kind: "string")]) therefore falls through to Custom(...), whose get_encoded_ref_type always returns CUSTOM_TYPE_NOT_SUPPORTED. The kind override provides no escape hatch because it routes through the same Custom path.

Impact

Structs or enums that contain bool fields or that want to encode a string member cannot use the #[type_hash] macro at all. Developers must hand-compute those type hashes. The macro fails closed (a compile error, not a wrong hash), so there is no correctness risk, only a functionality gap.

Recommendations

Add Bool and String variants to BasicType and the corresponding "bool" / "string" arms in from_str, with get_snip12_type_name returning "bool" / "string" and an empty

`get_encoded_ref_type` (the same pattern as the other basic types).

Remediation

This issue has been acknowledged by OpenZeppelin, and a fix was implemented in [PR #1739](#). They intend to merge these changes into the production branch.

3.12. Substring-based trait/hook detection produces false warnings

Target	with_components		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

All of the trait- and hook-presence checks in `add_per_component_warnings` and `validate_contract_module` operate on the stringified module body with `code.contains(...)` — for example, the `Votes` metadata check:

```
let snip12_metadata_implemented = code.contains("of SNIP12Metadata");
```

Because these are plain substring matches rather than AST-aware checks, they misfire in both directions:

- **False negatives** — a comment or unrelated identifier containing the pattern (e.g., `// TODO: implement ERC20HooksTrait`) suppresses a warning that should fire.
- **False positives** — a legitimate implementation written with a fully qualified trait path is not matched. For instance,

```
pub impl SNIP12MetadataImpl of
  openzeppelin_utils::cryptography::snip12::SNIP12Metadata {
  fn name() -> felt252 { "DAPP_NAME" }
  fn version() -> felt252 { "DAPP_VERSION" }
}
```

still triggers the `Votes` "SNIP12Metadata is missing" warning, since the body does not contain the exact substring of `SNIP12Metadata`.

Impact

Spurious or missing warnings may reduce developer trust in the macro's diagnostics.

Recommendations

Make these checks more syntax-aware (inspect the parsed impl items), or at least broaden them to recognize fully qualified trait paths and aliases.

Remediation

This issue has been acknowledged by OpenZeppelin, and a fix was implemented in [PR #1731](#). They intend to merge these changes into the production branch.

3.13. Enum variant encoding omits the colon used by common SNIP-12 implementations

Target	type_hash		
Category	Protocol Risks	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

For an enum, the macro encodes each variant as "Variant"(payload_types):

```
TypeVariant::Enum => {
    let tuple = maybe_tuple(&type_name)?;
    encoded_type.push_str(&format!("{}", name, tuple))
}
```

So the enum

```
enum A {
    X,
}
```

encodes to "A"("X"()). This matches the SNIP-12 text. However, several widely used SNIP-12 implementations — starknet.js, starknet-rs, and starknet.py — place a colon before the variant payload type (e.g., "A"("X": " ()")), deviating from the letter of the spec but forming the de-facto convention that wallets/signers implement.

Impact

A message whose type hash is computed by one of those libraries will not match the constant generated by this macro for the same enum, breaking signature interoperability with the dominant client tooling. There is no on-chain impact.

Recommendations

Decide explicitly whether to follow the spec (as the macro does at the time of writing) or the de-facto client convention. Either way, document the discrepancy prominently so integrators know which off-chain libraries their signatures are compatible with.

Remediation

This issue has been acknowledged by OpenZeppelin. OpenZeppelin notified us that they are aware of this situation and plan to follow what is specified in the SNIP-12 specification.

3.14. Tuple member encoding is not specified by SNIP-12

Target	type_hash		
Category	Protocol Risks	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The macro accepts tuple-typed members and encodes the tuple as a parenthesized, comma-separated list of the inner type names. For example,

```
struct A {
    X: (felt252, u128),
}
```

encodes to "A" ("X": "(felt, u128)"). SNIP-12 defines basic types, preset types, arrays, and user-defined struct/enum types, but it does not define an encoding for tuple types. The (type1, type2) spelling used here is therefore a macro-specific extension with no spec backing.

Impact

Because the spec does not define tuple encoding, a spec-compliant signer/wallet has no canonical way to reproduce the type hash of a tuple member, so any message using one is not interoperable with third-party tooling. There is no on-chain impact.

Recommendations

Document that tuple members are a nonstandard extension and describe the exact encoding used, or reject tuple members and require authors to model them as explicit SNIP-12 struct types. If the encoding is retained, cite the reference (implementation or spec discussion) it was derived from.

Remediation

This issue has been acknowledged by OpenZeppelin, and a fix was implemented in [PR #1744](#). They intend to merge these changes into the production branch.

3.15. The `assert_increase_allowance` function reverts when the current allowance is unlimited

Target	SafeERC20		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The `assert_increase_allowance` function reads the current allowance and approves the sum of it and `added_value`:

```
fn assert_increase_allowance(
    self: IERC20Dispatcher, spender: ContractAddress, added_value: u256,
) {
    let this = get_contract_address();
    let current = self.allowance(this, spender);
    assert(self.approve(spender, current + added_value),
        Errors::SAFE_ERC20_FAILED_OPERATION);
}
```

Many ERC-20 tokens (including OpenZeppelin's own ERC-20) treat `u256::MAX` as unlimited approval. When the current allowance is `u256::MAX`, `current + added_value` overflows and Cairo's checked arithmetic panics, reverting the call for any `added_value > 0`.

Impact

A helper intended to be a safe convenience wrapper reverts on a legitimate and common state (already unlimited allowance). Whether this is desirable is debatable — one can argue it is semantically correct to refuse to increase an already unlimited allowance — but the behavior is, at the time of writing, implicit (an arithmetic panic) rather than an intentional, documented choice.

Recommendations

Decide on the intended behavior and make it explicit; either short-circuit (return without a state change) when `current == u256::MAX`, or document that `assert_increase_allowance` intentionally reverts on unlimited allowances. Avoid relying on an opaque overflow panic to express the semantics.

Remediation

This issue has been acknowledged by OpenZeppelin. OpenZeppelin notified us that they prefer to keep the current semantics of this function for the sake of consistency with OpenZeppelin Solidity Contracts.

3.16. The ERC20FlashMint default configuration permits unbounded flash-mint amounts

Target	ERC20FlashMintComponent		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The default `FlashMintConfigTrait::max_flash_loan` allows minting up to the full remaining `u256` headroom of the token's supply:

```
fn max_flash_loan(
    self: @ComponentState<TContractState>, token: ContractAddress,
    total_supply: u256,
) -> u256 {
    let this = get_contract_address();
    if token != this {
        return 0;
    }
    Bounded::MAX - total_supply
}
```

With the default config, anyone can flash-mint an essentially arbitrary amount (bounded only by `u256` overflow) within a single transaction. Many downstream business logics — especially those that multiply token amounts as part of pricing or reward calculations — implicitly assume amounts stay within a sane range and can misbehave or overflow intermediate computations when confronted with a near-`u256::MAX` balance.

Impact

The integrating developer is ultimately responsible for choosing a sane cap, but the permissive default makes it easy to ship a contract that exposes the maximum possible flash-mint amount without a deliberate decision.

Recommendations

Guide developers toward a saner default — for example, ship a config that requires an explicit per-loan maximum, or surface a warning (via the `with_components` macro) when `ERC20FlashMint`

is used with the default `FlashMintConfigTrait` so the permissive behavior is an informed choice rather than an accident.

Remediation

This issue has been acknowledged by OpenZeppelin. OpenZeppelin notified us that they prefer to keep the current semantics of this function for the sake of consistency with OpenZeppelin Solidity Contracts.

3.17. Inconsistent supply-tracking extension strategy across components

Target	ERC1155SupplyComponent, ERC6909TokenSupplyComponent		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The two supply-tracking extensions require the integrator to wire a hook, but they expose that requirement through different shapes.

The ERC1155SupplyComponent provides an internal `after_update` whose name and signature mirror the `ERC1155HooksTrait::after_update` hook, so the integrator forwards the whole batch in one call:

```
fn after_update(
    ref self: ComponentState<TContractState>,
    from: ContractAddress,
    to: ContractAddress,
    token_ids: Span<u256>,
    values: Span<u256>,
) { ... }
```

The ERC6909TokenSupplyComponent instead exposes `update_token_supply`, which the integrator must call per token from either `before_update` or `after_update`, and it additionally requires an initializer call to register the SRC5 interface:

```
fn update_token_supply(
    ref self: ComponentState<TContractState>,
    sender: ContractAddress,
    receiver: ContractAddress,
    id: u256,
    amount: u256,
) { ... }
```

The two components thus differ in the hook they target, the function name, the call granularity (whole batch vs per token), and whether an initializer is needed — even though they solve the same problem (tracking per-ID total supply).

Impact

The divergent conventions increase the surface for integrator mistakes and make it harder to reason about the library uniformly, but neither shape is incorrect on its own.

Recommendations

Unless there is a clear rationale for the difference, unify the two extensions on a single convention (hook targeted, function naming, and call granularity) so integrators can apply one mental model across supply-tracking extensions.

Remediation

This issue has been acknowledged by OpenZeppelin.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Notable changes

We document the notable changes and additions within the scope of this audit.

ERC-20

- The ERC20Wrapper component, which wraps the underlying ERC-20 token, was added.
- ERC20FlashMint, an ERC-3156-compliant flash loan lender that only lends the token of the contract itself, was introduced.
- SafeERC20Dispatcher, which is useful on validating the return value from the methods of ERC-20 tokens, was implemented, and the existing calls to ERC-20 tokens were replaced to this dispatcher.

ERC-721

- The ERC721Consecutive component, which mints consecutive tokens at the time of deployment, was newly introduced.
- The ERC721 component was updated to allow a developer to customize the way a token's URI is built.
- The ERC721Wrapper component, which wraps the underlying ERC-721 token, was added.

ERC-1155

- The ERC1155Supply component, which helps to track the supply of ERC-1155 tokens, was added.
- The ERC1155 component was updated to allow a developer to customize the way a token's URI is built.

ERC-6909

- The ERC6909 component, which manages multiple tokens in a single contract in a simplified manner, was added.

Utils

- The `BitMap` struct that helps to manage a sparse bit array was introduced.
- The `lower_lookup` method was implemented in the `Trace` struct.

Macros

- The `with_components` macro was updated to support several components added, and new diagnoses for these components were introduced.
- The `type_hash` macro, which calculates the hash of given type based on the SNIP-12 specification, was introduced.
- The argument parsing mechanism was improved to handle more complex argument inputs.
- Helpers to use token streams provided by Scarb's macro API were implemented, and macros were updated to use these helpers.

5. Assessment Results

During our assessment on the scoped OpenZeppelin Cairo Contracts targets, we discovered 17 findings. No critical issues were found. Eleven findings were of low impact and the remaining findings were informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.